

MALE UAV Engine Digital Twin

Machine Learning System Documentation

Data | EDA | Features | Models | Accuracy | Deployment

Written for all team members - no ML background needed

Project: DRDO SIH-26054, AI-Enabled Real-Time Digital Twin for Aero Piston Engines

Engine: Rotax 912 ULS class | Model bundle: deployment-v6-max

Date: September 2026 | Version 1.0

1. Big Picture: What the ML System Does

The digital twin watches a UAV piston engine through 20 sensors, 10 times per second. The ML layer answers three questions, in order:

- 1. HEALTH WATCH: Is something wrong right now? (anomaly detection)
- 2. DOCTOR: If yes, which of the 8 known faults is it, and how sure are we? (fault classification)
- 3. COUNTDOWN: How many flight cycles until this engine fails? (RUL prediction)

A fourth piece, EXPLAINABILITY, justifies every diagnosis in plain sensor terms ("I say cooling failure because CHT is high while oil temperature stayed normal"). Everything runs live: edge device on the UAV does the fast health check, the ground station does the deep diagnosis.

2. Training Data: A Flight Simulator for Engines

Real failure data from military UAVs is classified and unavailable, so the project generates its own realistic data with textbook physics (generate_engine_data.py -> engine_data.csv). Think of it as a flight simulator for engines that also writes down the answer key.

2.1 Size and shape

Property	Value
Total sensor readings (rows)	45,812
Missions (flights)	100
Engines	5
Signals per reading	20 (RPM, CHT, EGT, oil P/T, fuel flow, vibration 1x/2x/0.5x, battery, throttle, altitude, ...)
Sampling rate	10 Hz (10 readings per second)

2.2 The physics is real

- ISA atmosphere model: air gets thin and cold with altitude, engines lose power (ICAO standard).
- Otto-cycle efficiency + Willans power curves, calibrated to the Rotax 912 ULS operator manual.
- Vibration model with 1x firing frequency, 2x harmonic, 0.5x sub-harmonic (how real misfires look on FFT).
- Thermal lag: metal heats/cools slowly (Newton cooling), oil warms slower than cylinder heads.

2.3 The 8 faults (FMEA-aligned)

Each faulty mission starts healthy, then one fault fades in mid-flight and grows slowly-then-fast (power-law curve, like real crack/wear growth), so the data contains early warning signs, not just the crash:

Fault	What happens in the data
misfire	RPM dips ~90, 1x vibration spikes, EGT wobbles
injector_abnormality	fuel flow +4.5 L/h, EGT +35 C, timing drifts
lubrication_issue	oil pressure falls toward 22 PSI, oil heats +22 C
cooling_degradation	CHT climbs toward 150 C redline
sensor_drift	measured CHT drifts +25 C while the true temperature is fine
combustion_instability	RPM hunting, EGT spikes, knock-like vibration
overheating	CHT +65 C, oil +28 C, pressure drops
abnormal_vibration	1x/2x/0.5x vibration all rise (prop/bearing)

2.4 Every row is labelled

- fault_type: which fault (or 'none').
- is_anomaly: 0 healthy / 1 abnormal.
- RUL: cycles left before failure (the countdown target).
- degradation_severity, health_index: how far gone the engine is (0-100).

3. EDA: Looking Before Learning

EDA (Exploratory Data Analysis) means interrogating the data before training, so we never train blind. Results live in `eda_report.json` plus 12 charts in `charts/`.

3.1 What we checked and found

Question	Finding	Why it matters
How much is faulty?	~31% anomalous rows	Balanced enough; no class-starvation panic
All 8 faults present?	Yes, ~1,500-2,000 rows each	Model can learn every fault
Sensors realistic?	Cruise CHT ~92 C (limit 150), EGT ~681 C, oil ~42 PSI	Matches Rotax manual: simulator trusted
What matters most?	CHT, oil temp/pressure, vibration harmonics, engine age	Guides feature design
Do faults separate?	PCA plot pulls fault clusters away from healthy	Learning the boundary is feasible

Rule of thumb: EDA is the quality certificate. If it showed zero overheating rows or a CHT of 500 C, every model trained afterwards would be garbage.

4. Honest Testing: Split by Whole Missions

Consecutive rows of one flight look almost identical. If mission #5 appears in BOTH training and testing, the model just memorizes it, scores 99%, and learns nothing. This trap is called data leakage.

Fix: split by whole missions (stratified group-by-mission, seed 42): ~65 missions train, ~15 validation (used only to pick the winner), ~20 locked test missions scored exactly once. 'Stratified' forces every fault into every split -- the old random split silently tested only 7 of 8 faults while claiming 8.

Split	Missions	Used for
Train	~65	Fit model parameters
Validation	~15	Pick best model + thresholds
Test (locked)	~20, all 8 faults	Final score, touched once

5. Feature Engineering: Engineer Thinking, as Numbers

Raw sensors alone score ~0.65 macro-F1. We add 44 physics-informed features (features.py): the same data, expressed the way an engineer thinks. All are computable live from the 20 CAN signals.

Feature idea	Example	Catches
Safety margins	150 - CHT, oil_pressure - 22	How close to the redline?
Vibration ratios	vib_1x / vibration	Misfire spikes 1x at any throttle
Timing deviation	timing - ideal curve	Combustion trouble
Decoupling residuals	cht_oil_ratio = CHT-rise / oil-rise	Lying sensor (CHT up, oil flat) vs real cooling fault (both up)
ISA corrections	sigma_isa, altitude-corrected EGT	Thin cold air is not a fault

No labels leak in: a feature may never use fault_type, RUL, or severity. RUL models additionally get elapsed cycle time -- you cannot predict 'time left' without 'time passed'.

6. The Three Models

6.1 Health watchdog + supervised head (two layers)

Layer 1 learns the shape of HEALTHY data only (Mahalanobis distance: mean + spread of 20 signals) and flags anything weird -- including a 9th fault nobody has ever seen. Layer 2 is a RandomForest-200 trained on labelled healthy/faulty rows (deployable signals only) that confirms known faults with a probability. Two layers, best of both: novelty coverage plus 0.89 F1 accuracy.

6.2 Fault doctor: RandomForest classifier, 300 trees (supervised)

300 random decision trees vote on which of the 8 faults it is -- ask 300 doctors, take the majority. Beat ExtraTrees, gradient boosting, voting ensembles and per-group specialists in a full validation battery (train_max.py). Outputs the fault plus confidence %. SHAP explainability (with an importance fallback) justifies each call.

6.3 Countdown: HistGradientBoosting RUL regressor (supervised)

Predicts cycles left before failure. Trees are built in sequence, each fixing the last one's errors -- excellent for smooth wear trends. Inputs: 44 features + elapsed cycles.

Stage	Model	Training labels used
Anomaly (novelty)	Mahalanobis (healthy only)	is_anomaly
Anomaly (known)	RandomForest-200 head	is_anomaly
Fault (8-way)	RandomForest-300, balanced	fault_type
RUL (number)	HistGradientBoosting-500	RUL

7. Accuracy: Honest Held-Out Numbers

Scored once on locked test missions (verify_final.py). 'Before' = previous pipeline on an easier 7-class test.

Task	Before	Now	Plain meaning
Anomaly F1 / AUC	0.72 / 0.89	0.89 / 0.98	Catches ~9 of 10 bad readings
Fault acc / macro-F1	0.82 / 0.66	0.83 / 0.83	Right ~83% on EVERY fault equally
RUL error / R2	10.7 / 0.88	6.6 / 0.94	'Dies in ~43 cycles' is off by ~7

7.1 Per-fault report card (test set)

Fault	F1	Reading
lubrication_issue	0.97	Excellent
abnormal_vibration	0.99	Excellent
injector_abnormality	0.95	Excellent
misfire	0.91	Excellent
combustion_instability	0.89	Good
overheating	0.82	Good
sensor_drift	0.63	Hard: looks like heat
cooling_degradation	0.52	Hardest: confused with overheating

The thermal trio (cooling vs overheating vs lying sensor) all look like 'hot CHT' -- that confusion is the genuine frontier, improved from 0.40 to 0.52+ by the decoupling features plus the RandomForest. RUL per fault is best on lubrication/injector/overheating (MAE ~2-3) and worst on combustion-instability (~19, noisy signature).

8. Live Serving (and Staying Honest)

- One bundle: `models/deployment_bundle.joblib` (`deployment-v5-physics`), loaded once by the FastAPI backend.
- One code path: `phm_pipeline.py` serves the CLI demo, Streamlit dashboard, and API identically.
- OOD abstention: inputs beyond the worst training fault are declined ('physical twin authoritative') instead of inventing a diagnosis.
- Edge vs ground: tiny Mahalanobis head (~13 ms) fits the 10 Hz onboard budget; big ensembles run async at GCS (`edge_benchmark.py`).

9. Run It Yourself

Goal	Command
Train + select (battery)	<code>python train_max.py</code>
Refit winners + ship	<code>python ship_max.py</code>
Prove test scores	<code>python verify_final.py</code>
One live CAN packet demo	<code>python inference.py</code>
Latency + size proof	<code>python edge_benchmark.py</code>
Backend API	<code>python -m uvicorn api:app --host 127.0.0.1 --port 8000</code>
Dashboards	<code>streamlit run dashboard.py</code> <code>gcs-app: npm run dev</code>
Whole stack (Windows)	<code>start.bat</code> / <code>stop.bat</code>

10. Limits and Next Steps

- Simulator, not fleet data: recalibrate on real Rotax telemetry when available.
- Thermal-trio confusion (cooling F1 0.49): add exhaust per-cylinder detail or temporal (multi-row) models.
- RUL on combustion-instability (MAE ~19): sequence models (LSTM/temporal CNN) are the natural upgrade.
- Live demo loop runs hotter than training data: covered today by OOD abstention; align sim physics next.
- Future: federated learning across fleet, ONNX edge export, per-cylinder EGT sensing.

11. Glossary (30-Second Definitions)

Term	Meaning
EDA	Exploring data with stats/plots before training.
RUL	Remaining Useful Life: cycles left before failure.
Anomaly	A reading that does not look healthy (fault unknown yet).
F1-score	Balance of catching faults vs false alarms (1.0 = perfect).
AUC	Chance a random fault scores worse than a random healthy row.
Macro-F1	F1 averaged equally over all 8 faults (no hiding weak ones).
MAE / RMSE / R2	Average error / big-error-penalizing error / trend fit (1.0 = perfect).
Leakage	Test data leaking into training; fakes high scores.
SHAP	Method attributing a prediction to input features ('why').
OOD abstention	Refusing to guess far outside training data.

12. File Map: Where Everything Lives

File	Role
generate_engine_data.py	Physics simulator -> engine_data.csv
features.py	44 physics features (train + serve identically)
train_max.py	Candidate battery, validation selection (test untouched)
ship_max.py	Refit winners, locked-test score, ship bundle v6
verify_final.py	Independent re-score of saved models
phm_pipeline.py	Shared inference path (CLI, dashboard, API)
models/deployment_bundle.joblib	Serving bundle (v5-physics)
inference.py	Single-packet CAN demo with explanation
api.py / dashboard.py / gcs-app	Backend, Streamlit GCS, Next.js 3D GCS
edge_benchmark.py	Latency/size evidence
eda_report.json, charts/	EDA evidence